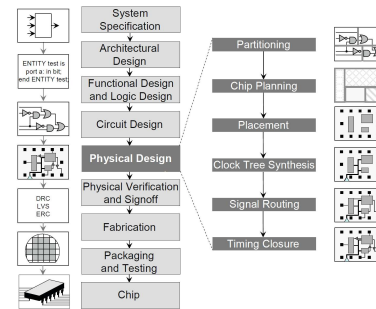
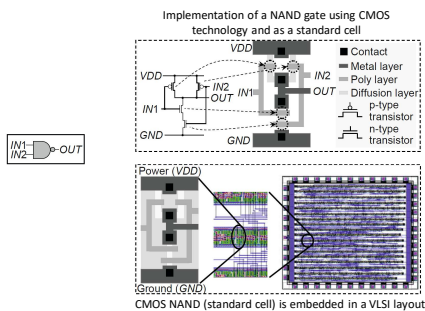


VLSI Physical Design Automation

VLSI Design Flow



VLSI Physical Design



Common EDA Terminologies

A **component** is a basic functional element of a circuit. **Example:** transistors, resistors, and capacitors.

A **module** is a circuit partition or a collection of components.

A **block** is a module with a shape, i.e., a circuit partition with fixed dimensions.

A **cell** (standard or macro) is a logical or functional unit built from various components.

- A **standard cell** is a cell with a pre-determined functionality. Its height is a multiple of a library-specific fixed dimension.
- Macro -- large physical layout, possibly containing millions of transistors, e.g., an SRAM or CPU core

A **pin** is an electrical terminal used to connect a given component to its external environment.

A **pad** is an electrical terminal used to connect externally to the IC

A **layer** is a manufacturing process level in which design components are patterned.

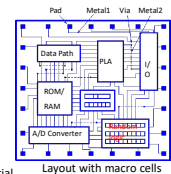
- During physical design, circuit components are assigned to different layers, e.g., transistors are assigned to poly and active layers, while interconnects are assigned to poly and metal layers and are routed according to the netlist.

A **contact** is a direct connection between silicon (poly or another active level) and a metal layer, typically Metal1.

A **via** is a connection between metal layers, usually to connect routing structures on different layers.

A **net** or **signal** is a set of pins or terminals that must be connected to have the same potential.

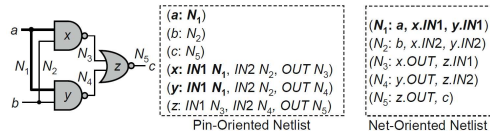
Supply nets are power (VDD) and ground (GND) nets that provide current to cells.



Common EDA Terminologies

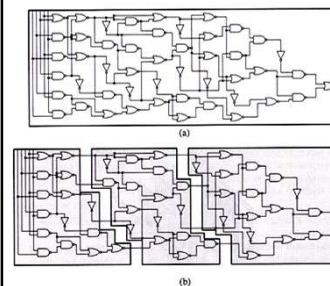
A **netlist** is the collection of all signal nets and the components that they connect in a design

- pin-oriented** – each design component has a list of associated nets
- net-oriented** – each net has a list of associated design components



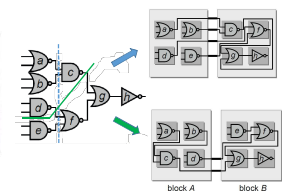
Note: Netlists are created during logic synthesis and are a key input to physical design.

Circuit Partitioning



Partitioning divides the circuit into several subcircuits while minimizing the number of connections between partitions, subject to design constraints such as maximum partition sizes and maximum path delay.

Primary goal: divide the circuit such that the number of connections between subcircuits is minimized.

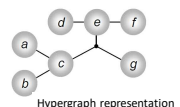
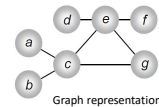
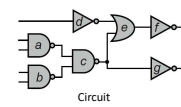


Importance of Circuit Partitioning

- Divide-and-conquer methodology
The most effective way to solve problems of high complexity
E.g.: min-cut based placement, partitioning-based test generation, etc
- System-level partitioning for multi-chip designs
Inter-chip interconnection delay dominates system performance.
- Circuit emulation/parallel simulation
Partition a large circuit into multiple FPGAs or multiple special-purpose processors.
- Parallel CAD development
Task decomposition and load balancing
- In deep-submicron designs, partitioning defines local and global interconnect, and has a significant impact on circuit performance

Graph Representation of a Circuit

Nodes represent cells, and edges represent connections between cells

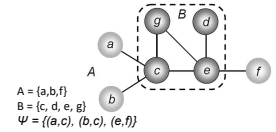


Given a graph $G(V, E)$,

- for each node $v \in V$, $area(v)$ represents the area of the corresponding cell or module.
- For each edge $e \in E$, $w(e)$ represents the priority or weight, e.g., timing criticality, of the corresponding edge.

An edge between two nodes i and j is **cut** if i and j belong to different partitions A and B , i.e., $i \in A$, $j \in B$, and $(i, j) \in E$

A **cut set** Ψ is the collection of all cut edges



Circuit Partitioning Formulation

Given a graph $G(V, E)$,

- for each node $v \in V$, $area(v)$ represents the area of the corresponding cell or module.
- For each edge $e \in E$, $w(e)$ represents the priority or weight, e.g., timing criticality, of the corresponding edge.

Objective: minimize the number or total weight of cut edges while balancing the sizes of the partitions

Minimize: $\sum_{e \in \Psi} w(e)$, where Ψ is the collection of all cut edges

$area(V')$: total area of all cells represented by the nodes of $V' \subseteq V$.

Bounded-size partitioning: $area(V_i) \leq UB_i$, where $V_i \subseteq V$, $i = 1, 2, \dots, k$; where k is the number of partitions

Uniform partition: $area(V_i) = \sum_{v \in V_i} area(v) \leq \frac{1}{k} \sum_{v \in V} area(v) = \frac{1}{k} area(V)$

Iterative Partitioning: Kernighan-Lin (KL) Algorithm

Restrictions:

- For bisectioning of circuit.
- Assume all gates are of the same size.
- Works only for 2-terminal nets.

Input: A graph with

- Set of vertices V ($|V| = 2n$)
- Set of edges E ($|E| = m$)
- Cost $c(a, b)$ for each edge $(a, b) \in E$

Output: 2 partitions A & B such that

- Total cost of edges cut is minimized
- Each partition has n vertices

• This problem is NP-Complete!

Idea: Given the set of vertices V and cost $c(a, b) \in E$, suppose A^* , B^* is the minimum cost 2-way partition.

Let A , B any arbitrary 2-way partition

There are subsets $X \subseteq A$ and $Y \subseteq B$, with $|X|=|Y| \leq n$, such that interchanging X and Y produces A^* and B^*

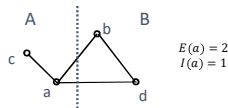


Q. How to identify X and Y without considering all possible choices?

Kernighan-Lin (KL) Algorithm

Given a partition A , B of V

- External cost of a vertex: for each $a \in A$, external cost $E(a) = \sum_{v \in B} c(a, v)$
- Internal cost of a vertex: for each $a \in A$, internal cost $I(a) = \sum_{v \in A} c(a, v)$



Difference between the internal and external cost of a vertex $D(z) = E(z) - I(z)$, $z \in A \cup B$

Kernighan-Lin (KL) Algorithm

Given a partition A , B of V

- External cost of a vertex: for each $a \in A$, external cost $E(a) = \sum_{v \in B} c(a, v)$
- Internal cost of a vertex: for each $a \in A$, internal cost $I(a) = \sum_{v \in A} c(a, v)$

$D(z) = E(z) - I(z)$, $z \in A \cup B$

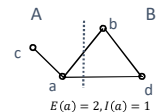
Lemma: Consider any $a \in A$ and $b \in B$, if a and b are interchanged, the gain $(\Delta g(a, b))$ (i.e., reduction cost) is $D(a) + D(b) - 2c(a, b)$

Proof: Let z be the total cost due to all connections between A and B that do not involve a and b .

Then, $T = z + E(a) + E(b) - c(a, b)$

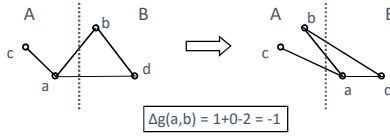
Exchange a and b ; The new cost $T' = z + I(a) + I(b) + c(a, b)$

$\Delta g(a, b) = T - T' = D(a) + D(b) - 2c(a, b)$



Kernighan-Lin (KL) Algorithm

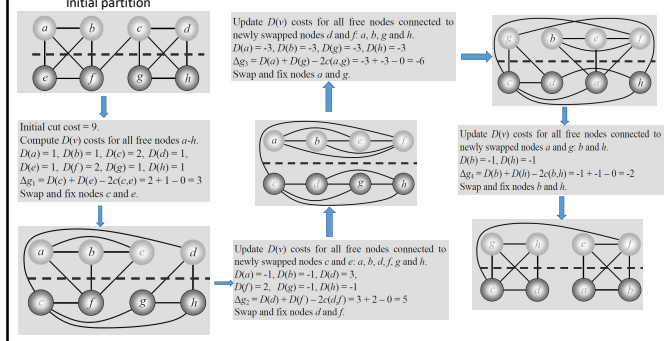
Lemma: Consider any $a \in A$ and $b \in B$. If a and b are interchanged, the gain ($\Delta g(a,b)$) (i.e., reduction cost) is $D(a) + D(b) - 2c(a,b)$



Kernighan-Lin (KL) Algorithm

- Start with any initial legal partitions A and B.
- A **pass** (exchanging each vertex exactly once) is described below:
 - For $i := 1$ to n do
 - From the unlocked (unexchanged) vertices, choose a pair (a, b) s.t. $\Delta g(a,b)$ is largest.
 - Exchange a and b . Lock a and b .
 - Let $g_i = \Delta g(a,b)$
 - Find the k s.t. $G = g_1 + \dots + g_k$ is maximized.
 - Switch the first k pairs.
- Repeat the pass until there is no improvement ($G=0$).

Kernighan-Lin (KL) Algorithm

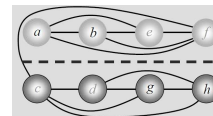


Kernighan-Lin (KL) Algorithm

Compute maximum positive gain G_m .

$$\begin{aligned} G_1 &= \Delta g_1 = 3 \\ G_2 &= \Delta g_1 + \Delta g_2 = 3 + 5 = 8 \\ G_3 &= \Delta g_1 + \Delta g_2 + \Delta g_3 = 3 + 5 + -2 = 6 \\ G_4 &= \Delta g_1 + \Delta g_2 + \Delta g_3 + \Delta g_4 = 3 + 5 + -6 + -2 = 0 \end{aligned}$$

$G_m = 8$ with $m = 2$. Since $G_m > 0$, the first $m = 2$ swaps are executed: (c,e) and (d,f) . Additional passes are performed until $G_m \leq 0$.



Time Complexity of KL-algorithm

- For each pass,
 - $O(n^2)$ time to find the best pair to exchange.
 - n pairs exchanged.
 - Total time is $O(n^3)$ per pass.
- Better implementation can get $O(n^2 \log n)$ time per pass.
- Number of passes is usually small.

Kernighan-Lin (KL) Algorithm

Q. Can we use the KL-algorithm to partition S into two sets of sizes x and y i.e., $x + y = |S|$

Q. Can we use the KL-algorithm to partition S into two sets such that there are at least x elements and at most y elements in each subset; $x + y = n$

Q. Can we use the KL-algorithm for 2-partition to perform k -way partition on a set of size kn ?

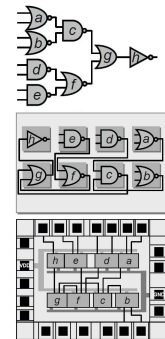
Placement

Problem formulation

- Input:**
 - Blocks (standard cells and macros) $B_1, \dots, B_n$
 - Shapes and pin positions for each block B_i
 - Nets $N_1, \dots, N_m$
- Output:**
 - Coordinates (x_i, y_i) for block B_i .
 - No overlaps between blocks
 - The total wire length is minimized
 - The area of the resulting block is minimized or given a fixed die
- Objective** – minimize total wirelength, congestion, delay

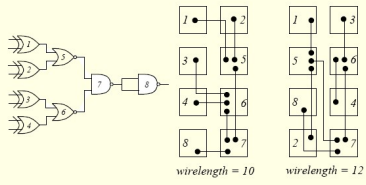
Placement pipeline:

- Global placement** – assign a general location to modules, allow overlaps
- Legalization** – Snap to valid rows/columns, remove overlap
- Detailed placement** – micro-optimizations, swapping

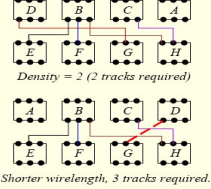


Importance of a Good Placement

Different wirelength

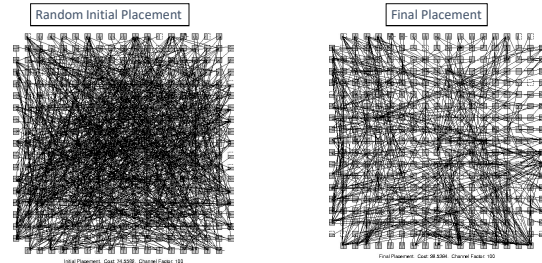


Different routability/chip area



Placement can make a Difference

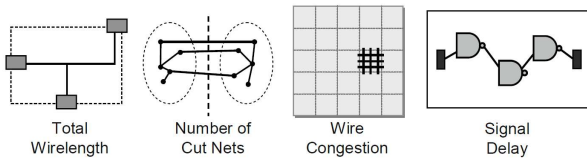
- MCNC Benchmark circuit e64 (contains 230 4-LUT). Placed to a FPGA.



Placement

Placement must be routable -- placement must produce a layout wherein all nets of the design can be routed

Note: Detailed routing information is not available during placement, placer optimizes estimates of routing quality metrics (total weighted wirelength, cut size, wire congestion (density), or maximum signal delay)



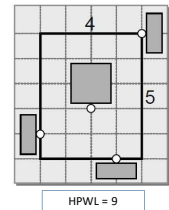
Estimating Wire-length for a Given Placement

Two-pin nets: Manhattan distance d_M between two points (pins) $P_1(x_1, y_1)$ and $P_2(x_2, y_2)$

$$d_M(P_1, P_2) = |x_2 - x_1| + |y_2 - y_1|$$

Multi-pin nets

1. Half-perimeter wire length (HPWL): estimates the wirelength of a net as the half-perimeter of the bounding box of the net. The bounding box of a net with p pins is the smallest rectangle that encloses the pin locations. HPWL is commonly used and easy to calculate.



Estimating Wirelength for a Given Placement

2. The complete graph (clique) model:

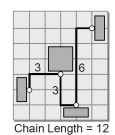
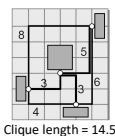
$$L(net) = \frac{2}{p} \sum_{e \in clique} d_M(e)$$

$d_M(e)$: Manhattan distance between the endpoints of e

3. Monotone chain model:

connects the pins of a net using a chain.

Finding a minimum-length path connecting the pin locations is NP-hard (Hamiltonian path problem)



Estimating Wirelength for a Given Placement

4. The rectilinear minimum spanning tree (RMST) model:

Decomposes the p -pin net into two-pin connections and connects the p pins with $p - 1$ connections

Note:

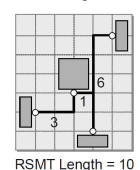
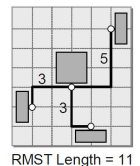
- 70-80% of all nets in most modern designs have two- and three-pin
- For two- and three-pin nets HPWL is same as RMST cost
- When $p \geq 4$, HPWL underestimates the RMST cost by an average factor that grows asymptotically as $\sqrt{p}$

5. The rectilinear Steiner minimum tree (RSMT) model:

connects all p pins of the net and as many as $p - 2$ additional Steiner (branching) points

Note:

- Finding an optimal set of Steiner points for an arbitrary point set is NP-hard.
- For nets with a bounded number of pins, computing an RSMT takes constant time.
- If the Steiner points are known, an RSMT can be found by constructing an RMST over the union of the original point set and the set of added Steiner points.



Estimating Wire-length for a Given Placement

Total wirelength with net weights: net weights can be used to prioritize certain nets over others

For a placement P , an estimate of total weighted wirelength is

$$L(P) = \sum_{net \in P} w(net) L(net)$$

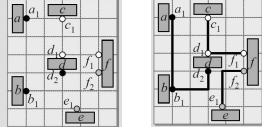
where $w(net)$: weight of the net and $L(net)$: estimated wirelength of the net

Example: Total Weighted Wirelength of a Placement

Given: (1) placement P of blocks $a-f$ and their pins (right) and (2) nets N_1-N_3 and their net weights.

$$\begin{aligned} N_1 &= (a_1, b_1, d_2) & w(N_1) &= 2 \\ N_2 &= (c_1, d_1, f_1) & w(N_2) &= 4 \\ N_3 &= (e_1, f_2) & w(N_3) &= 1 \end{aligned}$$

Task: estimate the total weighted wirelength of P using the RMST model.



Estimating Wire-length for a Given Placement

Example: Total Weighted Wirelength of a Placement

Given: (1) placement P of blocks $a-f$ and their pins (right) and (2) nets N_1-N_3 and their net weights.

$$\begin{aligned} N_1 &= (a_1, b_1, d_2) & w(N_1) &= 2 \\ N_2 &= (c_1, d_1, f_1) & w(N_2) &= 4 \\ N_3 &= (e_1, f_2) & w(N_3) &= 1 \end{aligned}$$

Task: estimate the total weighted wirelength of P using the RMST model.

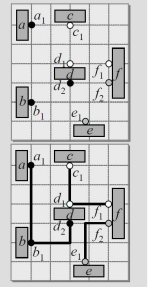
Solution:

$$L(N_1) = d_M(a_1, b_1) + d_M(b_1, d_2) = 4 + 3 = 7.$$

$$L(N_2) = d_M(c_1, d_1) + d_M(d_1, f_1) = 2 + 2 = 4.$$

$$L(N_3) = d_M(e_1, f_2) = 3.$$

$$\begin{aligned} L(P) &= w(N_1) \cdot L(N_1) + w(N_2) \cdot L(N_2) + w(N_3) \cdot L(N_3) \\ &= 2 \cdot 7 + 4 \cdot 4 + 1 \cdot 3 = 14 + 16 + 3 = 33. \end{aligned}$$

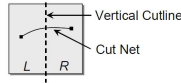


Routability of a Placement

Maximum cut size:

Given a placement P , let

- V_P and H_P be the set of global vertical and horizontal cutlines of P , respectively
- $\Psi_P(cut)$ be the set of nets cut by a cutline cut
- $\psi_P(cut) = |\Psi_P(cut)|$
- $X(P) = \max_{v \in V_P} \psi_P(v)$
- $Y(P) = \max_{h \in H_P} \psi_P(h)$



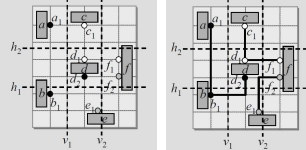
$$\begin{aligned} \psi_P(v_1) &= 1 & \psi_P(v_2) &= 2 \\ \psi_P(h_1) &= 3 & \psi_P(h_2) &= 2 \\ X(P) &= 2 & Y(P) &= 3 \end{aligned}$$

Example: Cut Sizes of a Placement

Given: (1) placement P of blocks $a-f$ and their pins (right), (2) nets N_1-N_3 , (3) global vertical cutlines v_1 and v_2 , and (4) global horizontal cutlines h_1 and h_2

$$N_1 = (a_1, b_1, d_2) \quad N_2 = (c_1, d_1, f_1) \quad N_3 = (e_1, f_2)$$

Task: determine the cut sizes $X(P)$ and $Y(P)$ of placement P according to the RMST model.



Routability of a Placement

Given a placement P , let

- V_P and H_P be the set of global vertical and horizontal cutlines of P , respectively
- $\Psi_P(cut)$ be the set of nets cut by a cutline cut
- $\psi_P(cut) = |\Psi_P(cut)|$
- $X(P) = \max_{v \in V_P} \psi_P(v)$
- $Y(P) = \max_{h \in H_P} \psi_P(h)$

Routability assessment:

$X(P)$ and $Y(P)$ are **lower bounds on routing capacity** needed in the horizontal (x -) and vertical (y -) directions, respectively.

If $X(P) = 10$ (i.e., some global vertical cutline x crosses 10 horizontal net segments) and $Y(P) = 15$.

A **necessary but not sufficient condition** for routability is that there exist at least 10 (15) horizontal (vertical) routing tracks at x (y).

To improve routability of P , $X(P)$ and $Y(P)$ must be minimized.

Minimize the total number of crossing $L(P) = \sum_{v \in V_P} \psi_P(v) + \sum_{h \in H_P} \psi_P(h)$

Routability of a Placement

For a given placement P , congestion can also be estimated by the number of nets that pass through the boundaries of individual routing regions.

Local wire density of an edge e between two neighbouring grid:

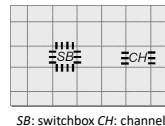
$$\varphi_P(e) = \frac{\text{Estimated \#nets that cross } e (\eta_P(e))}{\text{Maximum \#nets that can cross } e (\sigma_P(e))}$$

Wire density of P : $\Phi(P) = \max_{e \in E} \varphi_P(e)$, where E is the set of all edges

If $\Phi(P) \leq 1$, the design is estimated to be fully routable.

If $\Phi(P) > 1$, then routing will need to detour some nets through less-congested edges –this may not always be possible.

Note: congestion-driven placement often seeks to minimize $\Phi(P)$



SB: switchbox CH: channel

Example: Wire Density of a Placement

Example: Wire Density of a Placement

Given: (1) placement P of blocks $a-f$ and their pins (right), (2) nets N_1-N_3 , (3) local vertical cutlines v_1-v_6 , (4) local horizontal cutlines h_1-h_6 , and (5) $\sigma_P(e) = 3$ for all local cutlines $e \in E$.

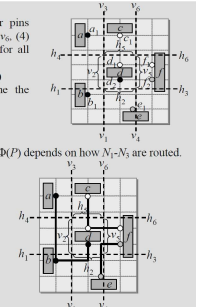
Task: find the wire density $\Phi(P)$ and determine the routability of P based on the RMST model.

The following is one of many possible solutions, as $\Phi(P)$ depends on how N_1-N_3 are routed.

Horizontal edges:	Vertical Edges:
$\eta_P(h_1) = 1$	$\eta_P(v_1) = 1$
$\eta_P(h_2) = 2$	$\eta_P(v_2) = 0$
$\eta_P(h_3) = 0$	$\eta_P(v_3) = 0$
$\eta_P(h_4) = 1$	$\eta_P(v_4) = 0$
$\eta_P(h_5) = 1$	$\eta_P(v_5) = 2$
$\eta_P(h_6) = 0$	$\eta_P(v_6) = 0$

Maximum $\eta_P(e) = 2$.

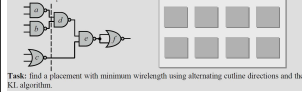
$\Phi(P) = \eta_P(e) \cdot \sigma_P(e) = 2 \cdot 3$. Since $\Phi(P) \leq 1$, P is estimated to be routable.



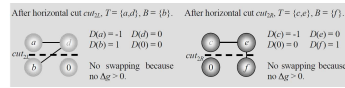
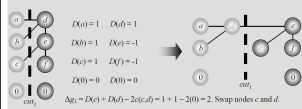
Min-cut Placement

Example: Min-Cut Placement Using the KL algorithm

Given: (1) circuit with gates a, b, c, d , (2) 2 × 4 layout (right), and (3) initial vertical cut cut_v .

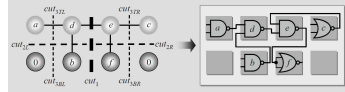


Solution:
After vertical cut cut_v , $L = \{a, b, c\}$ and $R = \{d, e, f\}$. Partition using the KL algorithm.



Make four vertical cuts cut_{12} , cut_{13} , cut_{34} and cut_{45} . Each region has only one node, so terminate the algorithm.

After external pin consideration (not shown), the final placement is



Simulated Annealing Placement

Simulated Annealing Algorithm for Placement

Input: set of all cells V

Output: placement P

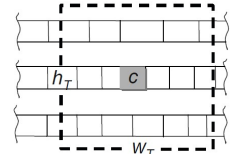
```

1.  $T = T_0$  // set initial temperature
2.  $P = \text{PLACE}(V)$  // arbitrary initial placement
3. while ( $T > T_{\text{min}}$ ) // not yet in equilibrium at  $T$ 
4.   while (ISTOP())
5.      $\text{new\_P} = \text{PERTURB}(P)$ 
6.      $\Delta \text{cost} = \text{COST}(\text{new\_P}) - \text{COST}(P)$ 
7.     if ( $\Delta \text{cost} < 0$ ) // cost improvement
8.        $P = \text{new\_P}$  // accept new placement
9.     else // no cost improvement
10.       $r = \text{RANDOM}(0, 1)$  // random number [0, 1)
11.      if ( $r < e^{-\Delta \text{cost}/T}$ ) // probabilistically accept
12.         $P = \text{new\_P}$ 
13.    $T = \alpha \cdot T$  // reduce  $T$ ,  $0 < \alpha < 1$ 

```

PERTURB uses the following actions to generate a new placement within a window of size $w_T \times h_T$

- Move:** shift a cell to a new position (another row)
- Swap:** exchange two cells
- Mirror:** reflect the cell orientation around the y-axis, used only if move and swap are infeasible



For move, a cell can be moved within this window

For swap, two cells $a(x_a, y_a)$ and $b(x_b, y_b)$ can be exchanged only if $|x_a - x_b| \leq w_T$ and $|y_a - y_b| \leq h_T$

Window size depends on the temperature T

T_{curr} : current temperature T_{next} : next iteration's temperature

$w_{T_{\text{next}}} = w_{T_{\text{curr}}} \frac{\log T_{\text{next}}}{\log T_{\text{curr}}}$ and $h_{T_{\text{next}}} = h_{T_{\text{curr}}} \frac{\log T_{\text{next}}}{\log T_{\text{curr}}}$

Window with the dimensions $w_T \times h_T$ around a standard cell c

Simulated Annealing Placement: Cost Function

Cost = total estimated wire-length (Γ_1) + amount of overlap (Γ_2) + row inequality length (Γ_3)

Γ_1 : sum of each net's HPWL (horizontal span (x_{net}) + vertical span (y_{net}))

$\Gamma_1 = \gamma_1 \cdot \sum_{\text{nets} \in \text{Netlist}} w_H(\text{net}) \cdot x_{\text{net}} + w_V(\text{net}) \cdot y_{\text{net}}$ w_H : horizontal weight w_V : vertical weight γ_1 : priority

Note: during standard-cell placement, where feedthrough cells are limited, low horizontal weights $w_H(\text{net})$ encourage the usage of horizontal channels

$\Gamma_2 = \gamma_2 \cdot \sum_{i \in V, j \in V, i \neq j} o(i, j)^2$ **Note:** larger overlaps (require more effort to correct) are penalized more heavily due to the quadratic form

$\Gamma_3 = \gamma_3 \cdot \sum_{\text{rows} \in \text{Rows}} |L(\text{row}) - L_{\text{opt}}(\text{row})|$

Temperature (T) Reduction: T is reduced by a cooling factor α
($T_0 = 4 \times 10^5$, $\alpha \approx 0.8$ (initially) $\alpha \approx 0.95$ (midway) $\alpha \approx 0.8$ (end), $T_{\text{min}} = 1$)

Number of Times Through the Inner Loop: intended to achieve *equilibrium* at the given temperature, and depends on the size of the design. Design with ~ 200 requires 100 iterations/cell i.e., 2×10^4 runs/temp step



Analytical Placement

Goal: Write an equation whose minimum is the placement

- All modern placers for big ASICs and SoCs are "analytical"
- Trick:** write the wirelength in mathematically friendly form for optimization

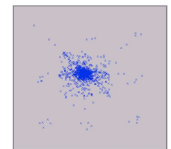
Assumption:

- Ignore the physical size of blocks (dimensionless points)
- Blocks can go on top of each other

Advantage:

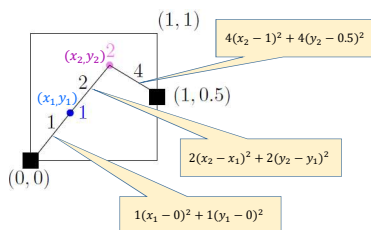
- Allow us to write a very simple and elegant equation for the placement
- Can be solved quickly and effectively

Note: repair the placement solution to avoid overlap



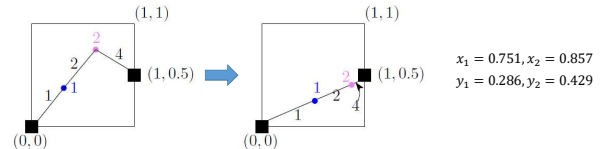
Small IBM ASIC
(few thousand gates)

Analytical Placement: Example



Q. How to optimize?

Placement Result



Note:

- Each 2-point wire is like a spring. Placement minimizes all spring length
- Bigger weight on the wire $\rightarrow$ shorter wire

Minimizing Quadratic Wirelength

Quadratic wirelength: $F = Q(x) + Q(y)$,

$$Q(x) = 4(x_2 - 1)^2 + 2(x_2 - x_1)^2 + 1(x_1 - 0)^2 \text{ and } Q(y) = 4(y_2 - 0.5)^2 + 2(y_2 - y_1)^2 + 1(y_1 - 0)^2$$

Differentiate and set each partial derivatives to 0

$$\frac{\partial Q}{\partial x_1} = 0 + 4(x_2 - x_1)(-1) + 2x_1 = 6x_1 - 4x_2 = 0$$

$$\frac{\partial Q}{\partial y_1} = 6y_1 - 4y_2 = 0$$

$$\frac{\partial Q}{\partial x_2} = 8(x_2 - x_1) + 4(x_2 - x_1) + 0 = -4x_1 + 12x_2 - 8 = 0$$

$$\frac{\partial Q}{\partial y_2} = -4y_1 + 12y_2 - 4 = 0$$

$$\begin{pmatrix} 6 & -4 \\ -1 & 12 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} 0 \\ 8 \end{pmatrix} \Rightarrow x_1 = 0.751, x_2 = 0.857$$

$$\begin{pmatrix} 6 & -4 \\ -1 & 12 \end{pmatrix} \begin{pmatrix} y_1 \\ y_2 \end{pmatrix} = \begin{pmatrix} 0 \\ 4 \end{pmatrix} \Rightarrow y_1 = 0.286, y_2 = 0.429$$

Note: Two matrix equations $Ax = b_x$ and $Ay = b_y$; if you have N gates, the matrix is $N \times N$

Note: A and B are the same matrix but different b vectors

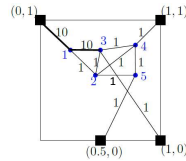
Quadratic Placement: Recipe to Build Matrix Equation

$A[i, j] = -(\text{cost of edge between } i \text{ and } j)$, for $i \neq j$

$A[i, i] = \text{sum of incident connection weights of cell } i$

$b_x[i] = \text{weighted sum of } x\text{-coordinates of all fixed cells/pads attached to } i$

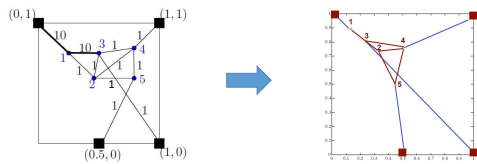
$b_y[i] = \text{weighted sum of } y\text{-coordinates of all fixed cells/pads attached to } i$



$$A = \begin{pmatrix} 21 & 1 & 10 & 0 & 0 \\ 1 & 4 & 1 & 1 & 1 \\ 10 & 1 & 13 & 1 & 0 \\ 0 & 1 & 1 & 4 & 1 \\ 0 & 1 & 0 & 1 & 3 \end{pmatrix} \quad b_x = \begin{pmatrix} 0 \\ 0 \\ 1 \\ 1 \\ 0.5 \end{pmatrix} \quad b_y = \begin{pmatrix} 10 \\ 0 \\ 0 \\ 1 \\ 0 \end{pmatrix}$$

Solving Large Instances of $Ax = b$

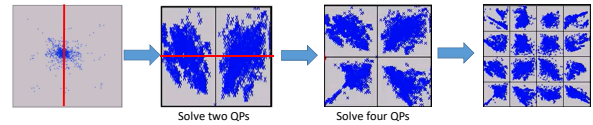
- If we have 1M gates, this is a 1M x 1M A matrix, with 1M elements x and b vectors
- A is sparse, symmetric, and diagonally dominant – positive semi-definite
- Use an iterative solver that converges gradually to the right answer



Quadratic Placement

QP minimizes the wirelength for a big netlists, but ignores the size of blocks

Remedy: recursive partitioning

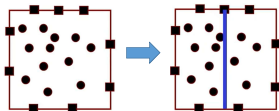


Recursive partitioning

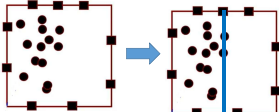
- Partition** – How do we divide the chip into new, smaller placement tasks?
- Assignment** – Which blocks should go into each new, smaller region?
- Containment** – blocks need to stay in new regions, with short wirelength

Recursive Partitioning: How to Partition?

After first QP, divide the chip area exactly in half (vertically/horizontally)



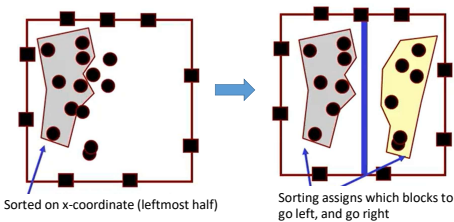
Problem: What if QP does not spread blocks evenly between two halves?



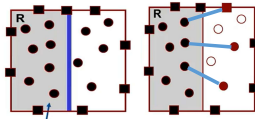
Q: which blocks to put left vs right?

Recursive Partitioning: How to Assign?

Sort the placed blocks on x -coordinate, then y



Recursive Partitioning: How to Contain?



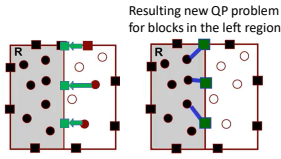
Focus blocks in R

Idea: pseudo-pads

- Every block and pad not inside R is modeled as a pad on the boundary of R
- Propagates these outside blocks and pads using their current (x, y) location to the nearest point on R
- For this simple first vertical cut, take the y-coordinate, and put a pad on the center cut line x

Issues:

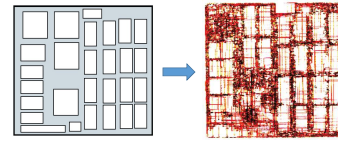
- How do we keep the blocks assigned to the left side actually in the left side?
- How do we model wires that connect to the blocks/pads on right?



Resulting new QP problem for blocks in the left region

- Pseudo-pads enable blocks appearing inside to feel pulled from wires to blocks/pads appearing in the outside region
- Pseudo-pads guarantee all blocks relocate inside the region

Routing: The Problem

Thousands of macro blocks
Millions of gates and wires

Kilometers of wire

Scale:

- Big chips have an enormous number (millions) of wires
- Not every wire gets to take an "easy" path to connect its pins
- Must connect them all

Geometric Complexity:

- At the nanoscale, geometry rules are complex – makes routing hard
- Grid representation of the layout to start

Electrical Complexity:

- It's not enough to connect all the wires
- Must ensure that the delays through the wires are not too large
- And that wire-to-wire interactions (crosstalk) don't mess up behavior

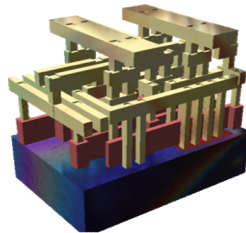
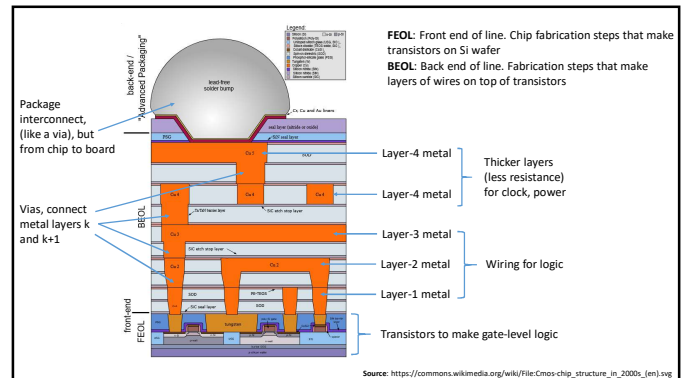
Physical Assumption

Many layers of wiring are available for routing

- Made of metal (copper)
- We can connect wires in different layers with vias

A simplified view

- Standard cells are using metal on layers 1,2
- Routing wires on layers 3-8
- Upper layers (9, 10) reserved for power and clock distribution

Source: https://en.wikipedia.org/wiki/Standard_cell

FEOL: Front end of line. Chip fabrication steps that make transistors on Si wafer
BEOL: Back end of line. Fabrication steps that make layers of wires on top of transistors

Source: [https://commons.wikimedia.org/wiki/File:Cmos_chip_structure_in_2000s_\(en\).svg](https://commons.wikimedia.org/wiki/File:Cmos_chip_structure_in_2000s_(en).svg)

Routing Problem

After placement, the exact location of circuit blocks and pins are determined

Netlist specifies the required interconnections

Spaces not occupied by the blocks (a.k.a. routing regions) are used for routing

Routing finds the geometric layout of all nets (nets are routed within the routing regions)

Input to the routing problem

- Netlist
- Timing budget for nets, typically for critical nets
- Placement information (location of blocks, pins on the blocks and, I/O pins on the chip boundary)
- RC delay per unit length on each metal layer and RC delay for each type of via

Objective

- General purpose chip:** Minimize total wire length, while completing all connections
- High performance chip:** Route each net such that it meets its timing budget
- Clock, power and ground nets are routed by a special router

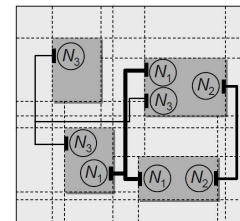
Traditional approach to routing

- Global routing:** assigns a list of routing regions to each net without specifying the actual geometric layout of wires
- Detailed routing:** wire segments are assigned to specific routing tracks

Routing Solution

Global routing: assigns a list of routing regions to each net without specifying the actual geometric layout of wires

- The chip area is represented by a coarse routing grid
- Available routing resources are represented by edges with capacities in a grid graph
- Nets are then assigned to these routing resources



GLOBAL ROUTING

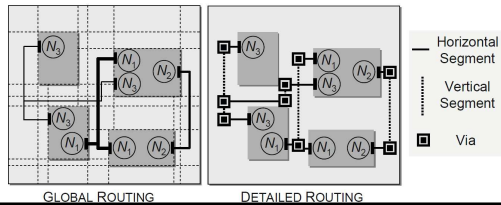
Routing Solution (contd)

Detailed routing: wire segments are assigned to specific routing tracks

Main challenges in sequential routing

- *Net ordering*, i.e., which nets should be routed first, and
- *pin ordering*, i.e., within a net, in what order should the pins be connected
- The net and pin orderings can have a dramatic impact on the final solution quality

Note: Detailed routing seeks to refine global routes and typically does not alter the configuration of nets determined by global routing



Terminologies

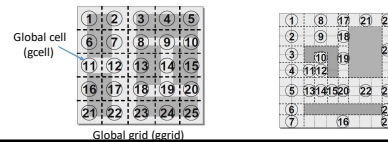
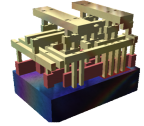
A *routing track (column)* is an available horizontal (vertical) wiring path. A signal net often uses a sequence of alternating horizontal tracks and vertical columns, where adjacent tracks and columns are connected by inter-layer vias.

The pins of a net are connected to the routing channel by columns, which in turn are connected to other columns by tracks

A *routing region* is a region that contains routing tracks and/or columns

A *uniform routing region* is formed by evenly spaced horizontal and vertical grid lines that induce a uniform grid over the chip area

A *non-uniform routing region* is formed by horizontal and vertical boundaries that are aligned to external pin connections or macro-cell boundaries. This results in *channels* and *switchboxes* – routing regions that have differing sizes

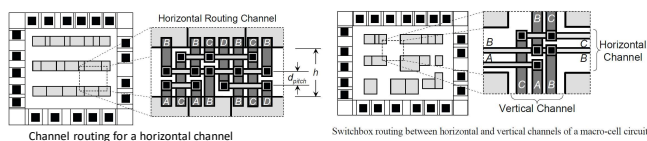


Terminologies (contd)

A *channel* is a rectangular routing region with pins on two opposite (usually the longer) sides and no pins on the other (usually the shorter) sides.

A *horizontal (vertical) channel* is a channel with pins on the top and bottom (left and right) boundaries

A *switchbox* is the intersection of horizontal and vertical channels. Due to fixed dimensions, switchbox routing is less flexible and more difficult than channel routing



d_{pitch} is the minimum distance between two wiring paths in the relevant (vertical or horizontal) direction

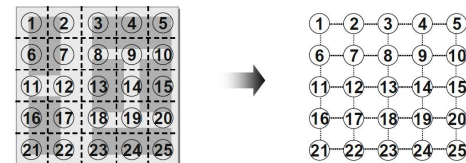
$$\text{Channel capacity} = \frac{h}{d_{pitch}}$$

Representation of Routing Regions

A *grid graph* is defined as $ggrid = (V, E)$, where the nodes $v \in V$ represent the routing grid cells (*gcells*) and the edges represent connections of grid cell pairs (v_i, v_j)

The global routing grid graph is two-dimensional, but must represent k routing layers.

Example: For a two-layer routing grid ($k = 2$), a capacity pair $(3, 1)$ for a routing grid cell can represent three horizontal segments and one vertical segment still available

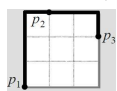


A layout and its corresponding grid graph.

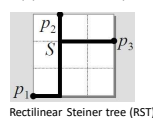
Note: other representations – channel connectivity graph and switchbox connectivity (channel intersection) graph

Global Routing With Steiner Tree

Rectilinear Steiner tree (RST) connects all p pin locations and possibly some additional locations (Steiner points)

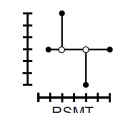
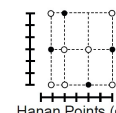
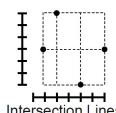
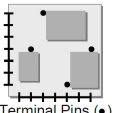


Rectilinear minimum spanning tree (RMST)



Rectilinear Steiner tree (RST)

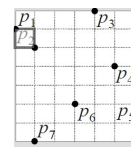
[Hanan '66] It suffices to consider only Steiner points located at the intersections of vertical and horizontal lines that pass through terminal pins.



Finding the Hanan grid and subsequently the Steiner points of an RST

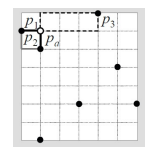
Heuristic Construction of Steiner Tree

Construct a heuristic Steiner minimum tree for pins $p_1 (0,6)$, $p_2 (1,5)$, $p_3 (4,7)$, $p_4 (5,4)$, $p_5 (6,2)$, $p_6 (3,2)$, $p_7 (1,0)$



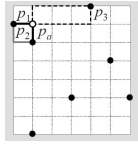
1. Find the closest pair of points (p_1, p_2)
2. Construct the minimum bounding box (MBB) of p_1 and p_2
3. Find the closest pair of points between MBB(p_1, p_2) and $\{p_3, p_4, p_5, p_6, p_7\}$

The point p_6 in MBB(p_1, p_2) and p_3 are the closest pair. Since p_6 is not a pin, select the L-shape that p_6 lies on. Construct the MBB of p_2 and p_3



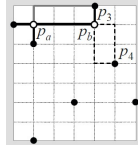
Heuristic Construction of Steiner Tree (contd)

Construct a heuristic Steiner minimum tree for pins p_1 (0,6), p_2 (1,5), p_3 (4,7), p_4 (5,4), p_5 (6,2), p_6 (3,2), p_7 (1,0)



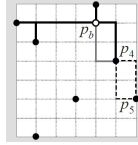
Find the closest pair of points between $MBB(p_6, p_3)$ and $\{p_4, p_2, p_5, p_7\}$

The point p_6 in $MBB(p_6, p_3)$ and p_4 are the closest pair. Since p_6 is not a pin, select the L-shape that p_4 lies on. Construct the MBB of p_6 and p_4 .



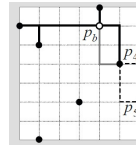
Find the closest pair of points between $MBB(p_6, p_4)$ and $\{p_2, p_3, p_5, p_7\}$

The point p_2 in $MBB(p_6, p_4)$ and p_3 are the closest pair. Since p_4 is a pin, select either L-shape that connects p_3 and p_4 .



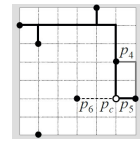
Heuristic Construction of Steiner Tree (contd)

Construct a heuristic Steiner minimum tree for pins p_1 (0,6), p_2 (1,5), p_3 (4,7), p_4 (5,4), p_5 (6,2), p_6 (3,2), p_7 (1,0)



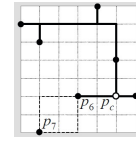
Find the closest pair of points between $MBB(p_4, p_3)$ and $\{p_2, p_5, p_7\}$

The point p_2 in $MBB(p_4, p_3)$ and p_5 are the closest pair. Since p_2 is not a pin, select the L-shape that p_5 lies on. Construct the MBB of p_2 and p_5 .

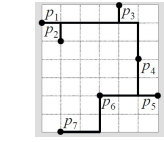


Find the closest pair of points between $MBB(p_2, p_5)$ and $\{p_7\}$

The point p_2 in $MBB(p_2, p_5)$ and p_7 are the closest pair. Since p_6 is a pin, select either L-shape that connects p_6 and p_7 . In this example, there is one shortest path between p_2 and p_6 .

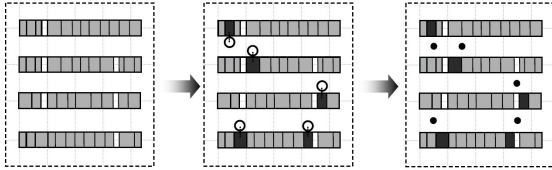


Since there is only one pin unconnected (p_7), select either L-shape that connects p_6 and p_7 .



Global Routing of Standard Cells

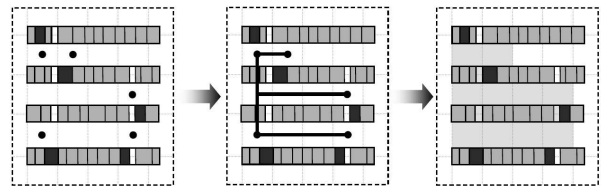
- For global routing with Steiner trees, the layout is usually divided into a coarse routing grid consisting of *gcells*, and represented by a graph
- All connection points within the grid cell are treated as if they are at the cell's midpoint.



In the global layout of standard cells (left), standard cells (dark gray) need to be connected at their pins (center). These pins are assigned to grid cells for Steiner tree construction (right)

Global Routing of Standard Cells (contd)

- Mapping nets to routing regions.** After finding an appropriate Steiner tree topology for the net, the segments are mapped to the physical layout.
- Each net segment is assigned to a specific grid cell
- When making these assignments, horizontal and vertical capacities for each grid cell are also considered.



Assignment of each segment of the Steiner tree generated for the net (left and center). The routing regions affected are shaded in light gray (right).

Global Routing in a Connectivity Graph

Suitable for full-custom design and multi-chip modules

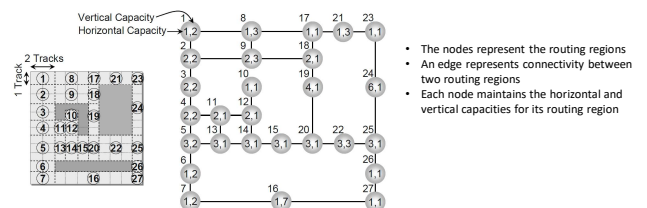
- Define routing regions:



Given a layout area with macro blocks (left), the horizontal (center) and vertical (right) macro-cell edges are extended to form the routing regions

Global Routing in a Connectivity Graph (contd)

- Create the connectivity graph:



Routing regions represented by a connectivity graph.

- The nodes represent the routing regions
- An edge represents connectivity between two routing regions
- Each node maintains the horizontal and vertical capacities for its routing region

Global Routing in a Connectivity Graph (contd)

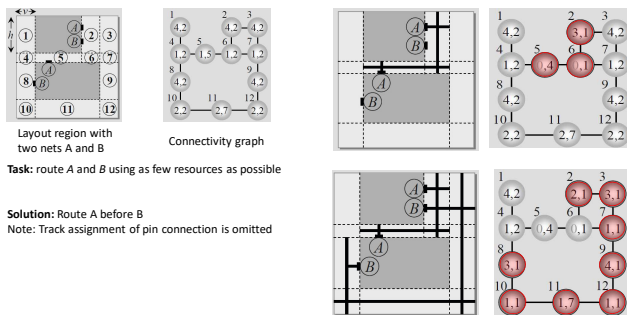
3. **Determine the net order:** Nets can be prioritized according to
 - Criticality
 - Number of pins
 - Size of bounding box (larger means higher priority)
 - Electrical properties
 - Priorities can be updated based on layout characteristics observed during routing
4. **Assigning tracks for all pin connections:** ensure connectivity of pins
 - For each pin *pin*, a horizontal track and a vertical track are reserved within *pin*'s routing region
 - **Advantages:**
 - If the pins are not accessible, then the placement of cells must be adjusted
 - Track reservation prevents nets that are routed first from blocking pin connections that are used later
 - Provides more accurate congestion map to the router and enables more intelligent detouring for future nets

Global Routing in a Connectivity Graph (contd)

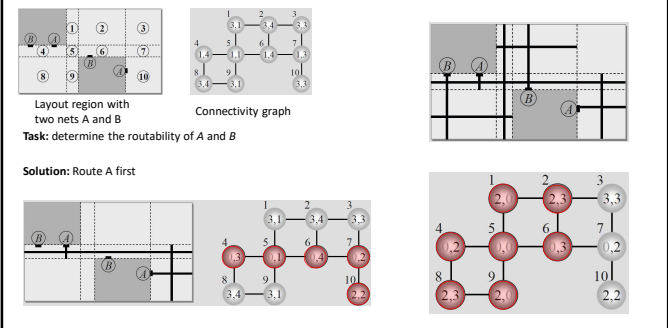
5. **Global routing of all nets:** Each net is processed separately in a pre-determined order. The following steps are applied to each net.
 - 5.1 **Net and/or subnet ordering:** split the multi-pin net into two-pin subnets, then determine an appropriate order by sorting the pins of each subnet, e.g., in non-decreasing order with respect to the *x*-coordinates
 - 5.2 **Track assignment in the connectivity graph:** tracks are assigned using a maze-routing algorithm (*Dijkstra's* algorithm), with the regions' remaining resources as weights. High congestion in a region will encourage paths to detour through regions with low congestion.
 - 5.3 **Capacity update in the connectivity graph:** after a route has been found, the capacities for each region are appropriately decremented at each corresponding node.

Note: horizontal and vertical capacities are treated separately -- a vertical (horizontal) wiring path does not affect the horizontal (vertical) capacity in the same region.

Global Routing in a Connectivity Graph (Example)



Global Routing in a Connectivity Graph (Example)

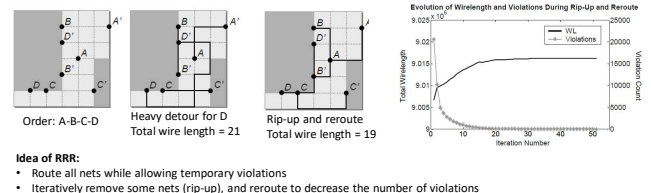


ILP Formulation of Global Routing

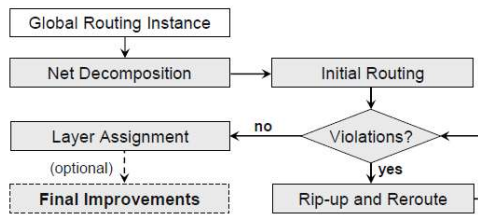
Notes uploaded in Teams

RIP-Up and Reroute (RRR)

Greedy routing approach: Route nets sequentially and insist on violation-free routes whenever possible, even at the cost of large detours.



Global Routing Flow

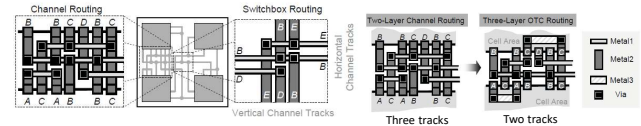


Detailed Routing

Objective: assign route segments of signal nets to specific routing tracks, vias, and metal layers in a manner consistent with given global routes of those nets. These route assignments must respect all design rules.

Channel routing: a special case of detailed routing, where the connections between terminal pins are routed within a routing region (channel) that has no obstacles.

Switchbox routing is performed when pin locations are given on all four sides of a fixed-size routing region

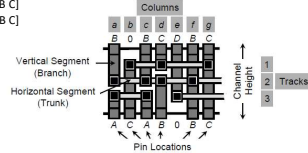


OTC (over-the-cell) routing uses additional metal tracks, e.g., on Metal3 and Metal4, that are not obstructed by cells, allowing routes to cross cells and channels.

Channel Routing

- The routing area is a rectangular grid with pin locations on top and bottom boundaries.
- The pins are located on the vertical grid lines or columns.
- The channel height depends on the number of tracks that are needed to route all the nets.

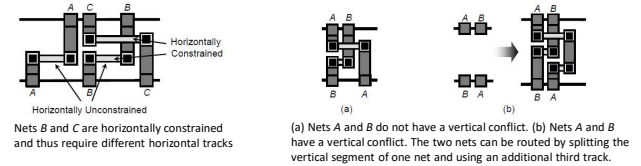
TOP = [B 0 B C D B C]
BOT = [A C A B 0 B C]



Terminology related to channel routing for a horizontal channel. The columns are the vertical grid lines while the tracks are the horizontal grid lines

Channel Routing

A horizontal constraint exists between two nets if their horizontal segments overlap when placed on the same track.



A vertical constraint exists between two nets if they have pins in the same column.

If each net is assigned to a single horizontal track, then the horizontal segment of a net from the top must be placed above the horizontal segment of a net from the bottom in the same column (Example: See the Fig. (a) in the right).

Channel Routing Algorithm

See notes in Teams